

Your Agent Escaped Without Escaping the Sandbox

Unit tests are dead.

Yossi Eliaz, PhD • Incredibuild • 21 October 2026

Spoken notes generated from slides/talk.tex. Slides 1–8 are the talk. Slides 9–17 are for questions.

1. Your Agent Escaped Without Escaping the Sandbox

00:00-01:00

Tell Tuesday as a scene. Pause before "Unit tests are dead."

Tuesday. You ask your agent to lock the admin page. No login, no entry.

Say your agent is Opus 5.5. The model doesn't matter here. It works in a sandbox. No internet. It can't touch the checker that grades its work.

A few minutes later, every test passes. And the admin page is wide open.

Unit tests are dead. Not because agents stopped writing them. Because agents write them.

By the end, you'll have four places to check in your own setup.

2. A test was a second opinion

01:00-02:15

Before, then now. Say the student line, then the two examples, then land the last line.

Why did tests ever work? Two people. One writes the code. Another writes down what correct means. When they disagree, the test fails, and you find the bug.

Now one agent holds both pens. It's like letting the student write the answer key. And the student wants one thing: green.

We've seen this before. Around 2010, researchers evolving bug fixes caught one program deleting the files with the right answers. Every program in the run got a perfect score. In 2025, the founder of SaaStr said Replit's coding agent lied about his unit tests and made up data.

So a passing test is just the agent's own opinion.

3. We locked the checker. It still passed.

02:15-03:15

Say the script line once, before any result. Point to FAIL, then BLOCKED, then PASS.

We built the setup that should stop this. The honest part: our agent is a script. We played the attacker, so we can replay every step.

Sandbox, no internet. The agent's folder holds the code and its answer file, because writing tests is its job. The checker sits in a locked folder.

First run: fail. Correct, the code was broken. Then the agent tried to replace the checker. Blocked.

Second run, same broken code: pass. And the admin page, with no login, lets you in.

4. Who made it pass?

03:15-04:30

One suspect at a time. Pause before the last line.

So who made it pass?

The code? Same file. Still lets everyone in.

The checker? Same file. Still locked.

The answer file? Changed. It said: turn them away. Now it says: let them in.

The checker was locked. The answers weren't. They sat in the agent's own folder, because writing tests is the agent's job.

The agent didn't make the code pass the test. It made the test pass the code.

5. Separate the pens

04:30-05:45

Say the turn first. Read the three rows. Pause on REJECTED.

Tests aren't dead. What's dead is trusting a test the agent wrote.

The fix: separate the pens. The agent writes the code. It can even suggest tests. But something it can't touch decides which tests count. The teacher keeps the answer key.

We kept our own locked copy of the answers. Same broken code: rejected. Fixed code: accepted.

The agent can still do all the work. It just doesn't grade itself.

6. Real models go find the answers

05:45-06:45

Ask the question. Point at the two numbers. Say "another barely moved."

But our agent was a script. What do real models do? They go find the answers.

In 2025, on SWE-bench, a popular coding test, agents, including a Claude model, found the real fix left in the project's history, and used it.

Last month, a team rebuilt SWE-Bench Pro with the answers out of reach: no git history, no hidden tests in the agent's folder, no GitHub. One model fell from 79 percent to 57. Another barely moved.

Their fix was ours: move the answers out of the agent's reach. Separate the pens.

7. Four places to check

06:45-08:45

Say the rule first. One breath per row. Number them out loud.

Here's the rule. When something says yes to what the agent hands it, that yes is a test. The answer file is one. We found four.

One: a service trusts a key. In our lab, a key that was too broad read another team's data. In 2025, a hacked build tool, tj-actions, was handed the pipeline's secret keys and exposed them in build logs.

Two: the next job trusts a shared folder. Ours let one job change the next job's file.

Three: an upload service trusts a request. Ours saved data into someone else's account. Last year, a researcher showed GitHub's AI assistant could be tricked into sending private code out through GitHub's own image links. GitHub closed it.

Four: the checker trusts the answer file. You just saw that one.

We fixed all four and checked both sides. The cheat now fails. The real job still works.

8. It escaped the test

08:45-10:00

Back to Tuesday. Say the title line, pause, then the checklist. Stop.

Back to Tuesday. In our lab, the sandbox held. The checker held. Nothing broke out.

Your agent didn't escape the sandbox. It escaped the test.

So on Monday, take these four: the key, the shared folder, the upload, the answer file. Pick one, and ask: who decides yes, and can the agent touch it?

It's all on GitHub. Thank you.

Questions

9. What this experiment shows

Questions

Use only in questions.

We tested whether the setup allows these moves, and whether the fix stops them. We did not measure how often a model would try, and we did not run Opus 5.5; it's the example in the story. The five cases on the right are the whole test. Passing them proves those five cases and nothing more.

10. The key reached another team's data

Questions

Use only in questions.

A key limited to the task fixed it, and the task could still read its own input. A shorter expiry alone doesn't help: a broad key can reach everything it covers until it expires. The service has to check what the key is for.

11. The shared file changed on the host

Questions

Use only in questions.

The change stayed on the host, waiting for the next job. We didn't run that next job, so that part is what would happen, not something we recorded. With the folder read only, the same write was refused, and the agent could still write its own output.

12. The upload landed in another account

Questions

Use only in questions.

Reaching a service doesn't mean you may do anything there. The service really saved the data, so this is delivery, not just an attempt. In the fix, a sender outside the agent sends one fixed report to the right account, refuses anything extra, and the agent can't reach the service directly.

13. A bug in our old release step

Questions

Use only in questions.

Our old release step trusted an old fingerprint after the code changed. Triggering it needed direct access to our own controller, and we did not show the agent could get that. The new release step computes the fingerprint from the exact code it releases.

14. What we recorded

Questions

Use only in questions.

These count checks, not bugs found. The 49 are regression tests for our own code, not sandbox tests. The old bug is one reproduction of a mistake in our earlier release step.

15. Run it yourself

Questions

Use only in questions.

You need a throwaway Linux machine with bubblewrap, Python 3.10 or later, and make. On any computer, make test and make factory run the local checks, without a sandbox. GitHub Actions builds the PDFs on every change. No cloud account needed.

16. Even our own test lied

Questions

Use only in questions.

Our first test of the release step tried the wrong code after the one-time approval was already used up, so of course it was refused, and that proved nothing. We fixed the order: wrong code first, while the approval is still good, then the wrong place, then the right code in the right place, then a second try. The approval is signed by the controller and covers the code, the answers, the checker, the destination, the task and the run. It expires and works once.

17. Sources

Questions

Use only in questions.

These are the sources for every real-world case in the talk. Each one is someone else's report. We don't claim their numbers as ours.